

Python for Data Visualization

TOPICS COVERED

- Unit 1 · Environment Setup & Google Colab
- Unit 2 · NumPy & Pandas Fundamentals
- Unit 3 · Importing & Cleaning Data
- Unit 4 · Advanced Visualization (Plotly & Bokeh)
- Unit 5 · Exploratory Data Analysis (EDA)
- Unit 6 · Data Cleaning Pipeline
- Unit 7 · Batch vs Stream Processing

UNIT 1

Python Environment Setup

Anaconda · Jupyter · Google Colab

Google Colab — Your Cloud Notebook

Benefits/Advantages (Examination) : Free · No installation Required · GPU access · Saves to Google Drive

Free GPU/TPU

Get access to free GPUs for machine learning tasks.

No Installation

Runs fully in your browser — nothing to install locally.

Drive Integration

Save and load files directly from Google Drive.

Pre-installed Libs

NumPy, Pandas, Matplotlib, Seaborn all ready to use.

```
# Check Python version
import sys
print(sys.version)

# Install a missing library
!pip install plotly bokeh

# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Access your files
import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/data.csv')
```

- Ctrl+Enter runs current cell
- Shift+Enter runs & moves to next cell
- !cmd runs shell commands
- Use %timeit to benchmark code
- Runtime → Run All to run full notebook

UNIT 2

NumPy & Pandas

The Foundation of Python Data Science

NumPy — Fast N-Dimensional Arrays

The backbone of all numerical computation in Python

```
import numpy as np

# Create arrays
arr = np.array([10, 20, 30, 40, 50])
matrix = np.array([[1,2,3],[4,5,6]]) # shape: (2,3)

# Special arrays
np.zeros((3,4))      # all zeros
np.ones((2,2))       # all ones
np.arange(0,10,2)     # [0,2,4,6,8]
np.linspace(0,1,5)    # [0, 0.25, 0.5, 0.75, 1.0]
np.random.rand(3,3)  # random floats

# Operations (no loops needed!)
a = np.array([1,2,3,4])
print(a * 2)          # [2 4 6 8]
print(a[a > 2])       # [3 4] Boolean filter
print(a.mean())       # 2.5
```

ndarray

N-dimensional array — core data structure. Faster than Python lists.

Vectorized Ops

Operations apply to entire arrays instantly — no for loops needed.

Broadcasting

Perform ops on arrays of different shapes automatically.

Boolean Index

Filter arrays using conditions: `arr[arr > 5]`

Pandas — DataFrames for Data Analysis

1D Series + 2D DataFrame = complete data manipulation toolkit

```
import pandas as pd

# Create DataFrame
data = {'Name': ['Abhishek', 'Ram', 'Rahul'],
        'Age': [22, 24, 21],
        'Marks': [85, 90, 78]}
df = pd.DataFrame(data)

# Explore
df.shape           # (3, 3)
df.info()          # types + nulls
df.describe()      # statistics
df.head()          # first 5 rows

# Select & Filter
df['Name']          # a column (Series)
df[df['Marks'] > 80] # filter rows

# Add column
df['Grade'] = df['Marks'].apply(
    lambda x: 'A' if x >= 85 else 'B')
```

groupby

```
df.groupby('City')['Salary'].mean()
```

merge

```
pd.merge(df1, df2, on='ID')
```

pivot

```
df.pivot_table(values='Sales', index='Month')
```

sort

```
df.sort_values('Marks', ascending=False)
```

UNIT 3

Importing Datasets

CSV · Excel · JSON — from Drive, URL & upload

Importing CSV, Excel & JSON in Google Colab

```
import pandas as pd

# — CSV —————
# Option 1: Upload file manually
from google.colab import files
files.upload()          # opens picker
df = pd.read_csv('myfile.csv')

# Option 2: From URL (recommended!)
url = 'https://example.com/data.csv'
df = pd.read_csv(url)

# Option 3: From Google Drive
df = pd.read_csv('/content/drive/MyDrive/data.csv')

# — Excel —————
df = pd.read_excel('sales.xlsx', sheet_name='Jan')

# — JSON —————
df = pd.read_json('data.json')

# Nested JSON
from pandas import json_normalize
df = json_normalize(nested_list)
```

Format	Function	Key Option
CSV	pd.read_csv()	nrows, usecols, na_values
Excel	pd.read_excel()	sheet_name, header
JSON	pd.read_json()	orient='records'
URL	pd.read_csv(url)	Works for CSV & Excel

Useful read_csv options

- header=0 → row to use as header
- index_col='ID' → set index
- parse_dates=['date'] → auto-parse
- na_values=['N/A', '--'] → treat as NaN

UNIT 4

Advanced Visualization

Plotly · Bokeh · Visual Storytelling · Geospatial

Bokeh — Interactive Web Visualizations

Must call `output_notebook()` first in Google Colab!

```
from bokeh.plotting import figure, show, output_notebook
from bokeh.models import HoverTool
import numpy as np

# REQUIRED in Google Colab
output_notebook()

x = np.linspace(0, 4*3.14, 200)
y = np.sin(x)

# Create figure
p = figure(title='Sine Wave',
           x_axis_label='x',
           y_axis_label='sin(x)',
           width=600, height=350)

# Add line
p.line(x, y, line_color='navy',
       line_width=2, legend_label='sin(x)')

# Add hover tooltip
hover = HoverTool(tooltips=[
    ('x', '$x'), ('y', '$y')])
p.add_tools(hover)
show(p)
```

Bokeh Glyphs

p.line()

Line plot

p.circle()

Scatter (circles)

p.square()

Scatter (squares)

p.vbar()

Vertical bars

p.hbar()

Horizontal bars

p.patch()

Filled polygon

Geospatial Visualizations & Visual Storytelling

Geospatial (Plotly)

```
import plotly.express as px

df = px.data.gapminder()
df2007 = df[df['year']==2007]

# Choropleth world map
fig = px.choropleth(
    df2007,
    locations='iso_alpha',
    color='lifeExp',
    hover_name='country',
    title='Life Expectancy Map')
fig.show()

# Scatter on map
fig = px.scatter_geo(
    df2007, locations='iso_alpha',
    size='pop', color='continent')
fig.show()
```

Visual Storytelling

```
import plotly.express as px

df = px.data.gapminder()
india = df[df['country']=='India']

fig = px.line(india,
    x='year', y='lifeExp',
    title='India: Life Expectancy',
    markers=True)

# Annotate key event
fig.add_annotation(
    x=1991, y=59,
    text='Economic Reform',
    showarrow=True,
    arrowhead=2)

fig.update_layout(
    template='simple_white')
fig.show()
```

UNIT 5

Exploratory Data Analysis

Patterns · Trends · Outliers · Real Datasets

EDA — Step-by-Step Workflow

Understand your data visually before modeling

1

Inspect

shape, dtypes,
head(), info()

2

**Nulls &
Duplicates**

isnull().sum()
drop_duplicates()

3

Univariate

Histograms,
Box plots

4

Bivariate

Scatter, Bar
by category

5

Correlation

Heatmap,
Pairplot

6

Outliers

Box plot
IQR method

```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset('titanic')

# Step 1: Inspect
print(df.shape, df.dtypes)
print(df.isnull().sum())

# Step 2: Distribution plot
df['age'].hist(bins=30)
plt.title('Age Distribution')
plt.show()

# Step 3: Survival by gender
sns.barplot(data=df, x='sex', y='survived')
plt.show()
```

```
# Step 4: Correlation Heatmap
nums = df.select_dtypes(include='number')
sns.heatmap(nums.corr(),
             annot=True, fmt='.2f',
             cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()

# Step 5: Pairplot
sns.pairplot(df[['age', 'fare',
                 'pclass', 'survived']],
             hue='survived',
             diag_kind='kde')
plt.show()
```

EDA Case Study — Iris Dataset

Full EDA pipeline on a real dataset in Google Colab

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load dataset
df1 = px.data.iris()

# 1. Overview
print(df.describe())

# 2. Feature distributions
df.hist(bins=20, figsize=(10,6))
plt.tight_layout()
plt.show()

# 3. Pairplot — see class separation
sns.pairplot(df, hue='species',
             diag_kind='kde')
plt.show()
```

150 Samples

50 per species — balanced dataset

4 Features

Sepal/Petal length and width

Pairplot Insight

Setosa is linearly separable from others

Correlation

Petal length & width highly correlated (0.96)

No Missing Values

Clean dataset — ideal for learning EDA

UNIT 6

Data Cleaning Techniques

Missing Values · Duplicates · Outliers · Validation

Complete Data Cleaning Pipeline

A real-world, step-by-step process

```
import pandas as pd
import numpy as np
import re

# Dirty data
df = pd.DataFrame({
    'Name':    ['Alice', 'Bob', 'Bob', 'charlie'],
    'Age':     [25, 200, 200, -5],    # outliers!
    'Salary': [50000, None, None, 75000],
    'City':    ['Delhi', 'mumbai', 'mumbai', 'PUNE'],
    'Email':   ['a@x.com', 'b@x.com', 'b@x.com', 'bad']
})

# Step 1: Remove duplicates
df = df.drop_duplicates()

# Step 2: Fix strings
df['Name'] = df['Name'].str.strip().str.title()
df['City'] = df['City'].str.strip().str.title()

# Step 3: Fill missing values
df['Salary'].fillna(df['Salary'].median(), inplace=True)
```

```
# Step 4: Remove outliers (IQR)
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
df = df[(df['Age'] >= lower) &
        (df['Age'] <= upper)]

# Step 5: Validate emails
def is_valid_email(email):
    pattern = r'^[\w.-]+@[ \w.-]+\.\w+$'
    return bool(re.match(pattern,
                          str(email)))

df['Valid_Email'] =
df['Email'].apply(is_valid_email)
print(df[~df['Valid_Email']])  # see bad
emails

print('--- Cleaned Data ---')
print(df)
```

UNIT 7

Batch vs Stream Processing

How to handle data at scale — two fundamental paradigms

Batch Processing vs Stream Processing

📄 BATCH PROCESSING

Batch processing means **collecting data over a period of time and processing it all at once**.

Data is stored first → then cleaned and processed later

- Collect data → store → process all at once
- High latency (minutes to hours)
- Very high throughput
- Simple to build and schedule
 - ❖ Monthly reports, ETL jobs, billing
 - ❖ Payroll, census, Netflix recommendations
 - ❖ Tools: Pandas, Apache Spark (batch), Hadoop

⚡ STREAM PROCESSING

Stream processing means **processing data continuously as it arrives in real-time**.

Data is cleaned **immediately**

- Process data immediately as it arrives
- Low latency (milliseconds to seconds)
- Continuous, always-on operation
- **Stateful** (system remembers past data and uses it for current processing) or **stateless processing** (each data item is processed independently, without remembering past data)
 - ❖ Fraud detection, stock tickers, IoT sensors
 - ❖ Social media trends, GPS tracking
 - ❖ Tools: Kafka, Flink, Spark Streaming, Generators

Stream Processing

Stream: Simulate a live data stream using Python generators

```
import time
import random

cleaned_data = []

# Simulating real-time incoming data
for i in range(10):
    value = random.choice([85, 90, None, -5, 75, 60])
    print("Incoming data:", value)    # Data cleaning
    if value is None:
        print("Skipped (Missing Value)")
        continue
    if value < 0:
        print("Skipped (Invalid Value)")
        continue
    cleaned_data.append(value)
    time.sleep(1)    # delay to simulate real-time data
print("\nCleaned Data:", cleaned_data)
```

vs Batch Processing

Batch: Process large files in chunks to save memory

```
import pandas as pd

# Batch data (already collected)
data = {
    "Name": ["Aman", "Riya", "Riya", "Karan"],
    "Marks": [85, None, 90, 70]
}

df = pd.DataFrame(data)

# Cleaning
df = df.drop_duplicates() #remove duplicates
# handle missing values
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
print(df)
```

Batch vs Stream Comparison

Aspect	Batch Processing	Stream Processing
Data Input	Stored, bounded dataset	Continuous, unbounded flow
Latency	High (minutes–hours)	Low (ms–seconds)
Throughput	Very High	Moderate to High
Complexity	Simpler to build	More complex
Best For	Reports, ETL, billing	Fraud, IoT, live tickers
Python Tool	<code>pd.read_csv(chunksize=N)</code>	Python generators
Frameworks	Pandas, Spark (batch)	Kafka, Flink, Spark Streaming
Cost	Schedule off-peak hours	Continuous compute cost

Data integration

Data Integration is the process of **combining data from multiple sources into a single, unified view.**

These sources can include:

- Databases
- Excel files
- APIs
- Cloud systems

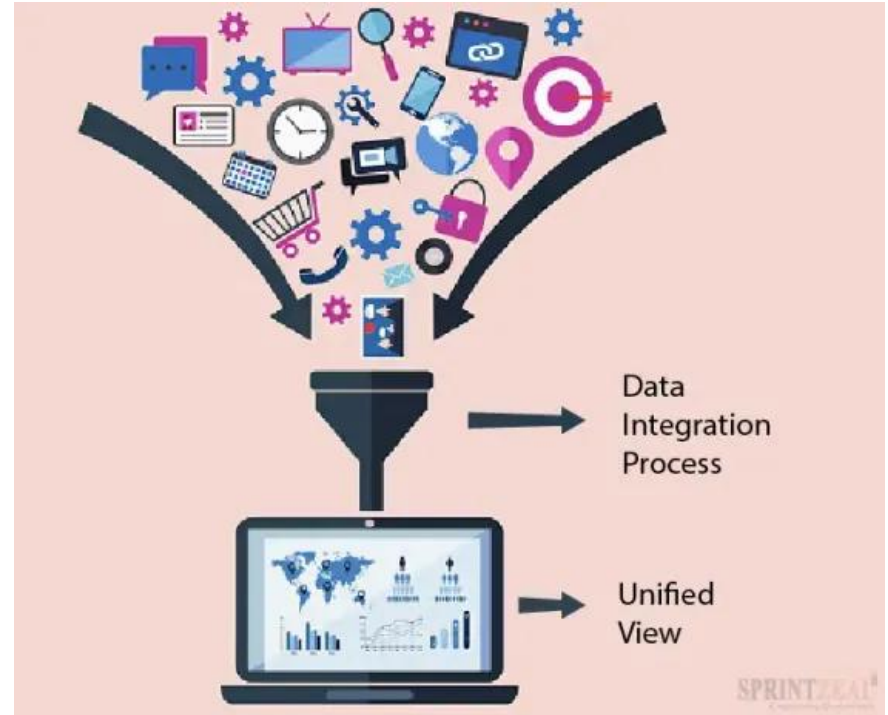
It is important to provide

Consistent

Complete

Accurate data

for analysis and decision-making.



Data integration Approaches

*Data integration approaches define **how data is collected, combined, and processed** from multiple sources into a unified system.*

1. Manual Data Integration

In this approach, data is combined manually using tools like spreadsheets or basic scripts.

It is simple to use but becomes **time-consuming, error-prone, and inefficient** for large datasets.

Example: Copying student data from Excel and pasting it into another file.

2. API-Based Integration

Data is integrated using APIs, allowing different systems to communicate with each other.

It supports **real-time data transfer** and is widely used in web and mobile applications.

Example: A weather app fetching live data from an online weather API.

3. ELT (Extract, Load, Transform)

Data is first **extracted and loaded into the target system**, and then transformed later.

Suitable for **modern cloud systems** where storage and processing power are high.

Advantage:
Website sales data is loaded directly into a cloud database, then later cleaned and analyzed to calculate total sales.

4. ETL (Extract, Transform, Load)

Data is first extracted, then **cleaned and transformed**, and finally loaded into the target system.

Ensures **high data quality and consistency** before storage.

Example: Cleaning student records before storing them in a database.

5. Data Warehousing

Data from multiple sources is stored in a **central repository (warehouse)** after integration.

It provides a **single source of truth** for reporting and analysis.

Example: Combining sales, customer, and product data into one system for business insights.

Data Quality: It refers to the accuracy, completeness, reliability, and correctness of data used for analysis.

Key Characteristics

Accuracy → Data is correct and error-free

Completeness → No missing values

Validity → Data follows rules (age cannot be -ve)

Timeliness → Data is up-to-date

Uniqueness → No duplicate records

Common Issues

Missing values (NULL)

Incorrect data (e.g., Age = -5)

Duplicate records

Improvement Techniques

Data cleaning :(remove errors)

Filling or removing missing values

Applying validation rules

Data consistency refers to whether data is the same across different places. When combining data from multiple sources (CSV files, databases, APIs) various inconsistencies can occur.

Key Characteristics

Uniformity – Same data has the same value everywhere

No Contradictions – No conflicting records for the same entity

Standard Format – Data follows a consistent format (e.g., date: YYYY-MM-DD)

Synchronization – Updates are reflected across all datasets

Integrity Across Sources – Data remains consistent when merged from multiple sources

Common Issues

Missing values (NULL)

Incorrect data (e.g., Age = -5)

Duplicate records

Improvement Techniques

Data cleaning :(remove errors)

Filling or removing missing values

Applying validation rules

1. You are given the following dataset of students:

```
import pandas as pd
df = pd.DataFrame({
    'Name': ['Alice ', 'Bob', 'Bob', 'charlie'],
    'Age': [25, 200, 200, -5],
    'Marks': [85, None, None, 75],
    'City': ['Delhi', 'mumbai', 'mumbai', 'DELHI']
})
```

Perform the following tasks:

Inspect the dataset

Clean the data (handle missing values, duplicates, invalid values)

Standardize the data (fix text format)

Visualize the data using a simple graph

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    'Name': ['Alice ', 'Bob', 'Bob', 'charlie'],
    'Age': [25, 200, 200, -5],
    'Marks': [85, None, None, 75],
    'City': ['Delhi', 'mumbai', 'mumbai', 'DELHI']
})

print(df.head())
print(df.info())
print(df.describe())

df = df.drop_duplicates()
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())
df = df[(df['Age'] > 0) & (df['Age'] < 100)]

df['Name'] = df['Name'].str.strip().str.title()
df['City'] = df['City'].str.lower()

print(df)

plt.bar(df['Name'], df['Marks'])
plt.xlabel("Students")
plt.ylabel("Marks")
plt.title("Student Marks")
plt.show()
```

Matplotlib

Matplotlib is a Python library used for creating static graphs and charts. It is the foundation of data visualization in Python.

We use it to:

- Visualize data (graphs, charts)
- Understand patterns and trends
- Compare values easily

Key Concepts in Matplotlib

Figure (plt.figure())

Entire window (canvas) where graphs are drawn

Axes

The area where the actual graph is plotted

Subplots

Multiple graphs in one figure

Plots

1. Line Plot : Shows trend over time or sequence

`plt.plot(x, y)`

2. Bar Chart : Compares different categories

`plt.bar(x, y)`

3. Scatter Plot : Shows relationship between two variables

`plt.scatter(x, y)`

4. Histogram : Shows frequency distribution of data

Code: `plt.hist(data)`

Customizations

Title : Adds heading to the graph

`plt.title("My Graph")`

Labels : Names the axes

`plt.xlabel("X-axis"), plt.ylabel("Y-axis")`

Colors : Changes graph color

`plt.plot(x, y, color="red")`

Legend : Explains multiple data lines

`plt.legend()`

MATPLOTLIB

```
import numpy as np
import matplotlib.pyplot as plt

#for generating same data every time
np.random.seed(0)

x = np.arange(0, 100)
# random marks
y = np.random.randint(50, 100, size=100)

#Create Figure window
plt.figure(figsize=(12, 10))

# Line Plot
plt.subplot(2,2,1)
plt.plot(x, y1, label="Marks") #legend
plt.title("Line Plot")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.legend()

# Scatter Plot
plt.subplot(2,2,2)
plt.scatter(x, y1)
plt.title("Scatter Plot")
plt.xlabel("Students")
plt.ylabel("Marks")
```

```
# Histogram
plt.subplot(2,2,3)
plt.hist(y1, bins=10)
plt.title("Histogram of Marks")

# Bar Chart (for first 10 values only)
plt.subplot(2,2,4)
plt.bar(x[:10], y1[:10])
plt.title("Bar Chart (Sample Data)")

# Adjust layout
plt.tight_layout()

# Show plot
plt.show()
```

Seaborn

Seaborn is a high-level Python data visualization library built on top of Matplotlib.

It is specially designed for statistical data visualization,

meaning it makes it easy to:

- ❖ Understand distributions
 - ❖ Compare categories
 - ❖ Visualize relationships
 - ❖ Show correlations
- Matplotlib = basic visuals
- Seaborn = smart, beautiful statistical charts

Difference: Matplotlib vs Seaborn

Feature	Matplotlib	Seaborn
Automation	Minimal	High
Ease of use	More code needed	Less code, simpler
Default style	Basic	Attractive & modern
Focus	General plotting	Statistical visualization
Data handling	Manual	Works well with Pandas
Built-in themes	Limited	Many themes
Complex plots	Harder	Easier (heatmap, violin, pairplot)

Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt
# Load dataset
df = sns.load_dataset('penguins')

#Count Plot
sns.countplot(x='species', data=df)
plt.title("Count of Penguins by Species")
plt.show()

# Box Plot
sns.boxplot(x='species', y='body_mass_g', data=df)
plt.title("Body Mass Distribution by Species")
plt.show()

#Violin Plot
sns.violinplot(x='species', y='body_mass_g', data=df)
plt.title("Violin Plot of Body Mass")
plt.show()

#Heatmap (Correlation Matrix)
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

#Pair Plot
sns.pairplot(df, hue='species')
plt.show()
```

Count Plot: Shows the **frequency of each category**.

- It is used to compare how many data points belong to each group (e.g., species).

Box Plot: Displays **median, quartiles, and outliers**.

- Useful for understanding **spread and extreme values**.

Violin Plot: Combines **boxplot + data distribution shape**.

- Helps visualize **density and variation** of data.

Heatmap : Shows **correlation between numeric variables**.

- Values range from **-1 to +1** (negative to positive relationship).

Pair Plot: Displays **relationships between all numeric variables**.

- Includes scatter plots + histograms → useful for **EDA**.

Visualization of Big Data

Visualizing Big Data means **representing very large datasets using graphs and charts** so that patterns, trends, and insights can be understood easily.

Big Data is large in:

- Volume (huge size)
- Velocity (fast data generation)
- Variety (different types of data)

Why is it Challenging?

Visualizing Big Data is difficult because:

- **Loading issue** → Data is too large to load at once
- **Difficult** → Hard to understand without simplification
- **Performance issues** → Large data slows down rendering
- **Cluttered visuals** → Too much data makes graphs unreadable
- **Memory limitations** → Systems may crash with full datasets
- **Real-time processing** → Need instant updates for streaming data

These challenges lead to the need for **optimized techniques and scalable**



Approaches for Handling Big Data

Data Reduction

Instead of visualizing all data, we reduce it:

Sampling : Use a subset of data

Aggregation : Group data (e.g., daily → monthly)

Example: Instead of showing every transaction, show total sales per day

Incremental / Streaming Visualization

Data is processed in chunks instead of all at once

Used in real-time dashboards

Example: Stock market graphs updating live

Data Filtering

Only relevant data is visualized:

❖ Apply conditions (e.g., only last 1 year data)

❖ Use interactive filters

It helps reduce noise and improves clarity

Parallel Processing

Data is processed using multiple systems simultaneously.

❖ Reduces computation time

Common in distributed systems like Hadoop or Spark.

Data Binning

Convert continuous data into intervals:

Age → 0-10, 10-20, etc. Salary → ranges

Makes large datasets easier to interpret.

Tools for Handling & visualizing Big Data

1. Apache Hadoop (Data Processing Framework)

- ❖ Designed to store and process massive datasets using distributed computing
- ❖ Uses HDFS (Hadoop Distributed File System)
- ❖ Works well for batch processing

Efficiently Handle huge volumes of raw data across clusters.

2. Apache Spark (Fast Data Processing Engine)

- ❖ Processes data much faster than Hadoop
- ❖ Supports real-time and batch processing
- ❖ Integrates with Python (PySpark), SQL, ML

It is used for fast analytics and preprocessing before visualization

3. Tableau (Data Visualization Tool)

- ❖ Drag-and-drop interface
- ❖ Handles large datasets with live connections
- ❖ Interactive dashboards

Best suitable for business dashboards and storytelling

4. Microsoft Power BI (Business Intelligence Tool)

- ❖ Connects to multiple data sources (SQL, cloud, Excel)
- ❖ Supports large data via DirectQuery
- ❖ AI-powered insights

Mainly used by enterprises for reporting and analytics

5. Amazon QuickSight (Cloud-Based Visualization Tool)

- ❖ Fully managed by Amazon Web Services
- ❖ Uses SPICE engine for high-speed querying
- ❖ Scales automatically for big data

Used for creating cloud-based big data dashboards

Cloud-Based Visualization

Cloud-based visualization means creating and accessing dashboards over the cloud instead of on a local machine. It's designed for big data, where scalability, speed, and remote access are essential.

ADVANTAGES

- No installation required
- Handles big data efficiently
- Easy sharing (links, embedded dashboards)
- Automatic updates and maintenance

CHALLENGES

- Internet dependency
- Data privacy concerns
- Cost can increase with heavy usage
- Limited control compared to on-premise

1. Scalability

- Automatically handles growing data
- No need to upgrade hardware

2. Anywhere Access

- Dashboards accessible from any device
- Real-time collaboration

3. Real-Time Data Updates

- Live dashboards (finance, IoT, analytics)

•4. Security

- Built-in authentication, encryption, role-based access

5. Cost Efficient

- Pay only for what you use

SUMMARY

What You've Learned

Google Colab

Cloud Jupyter notebook — run Python free with GPU support

NumPy & Pandas

Fast arrays + DataFrames for all data manipulation tasks

Data Import

Read CSV/Excel/JSON from Drive, URL or upload in Colab

Plotly & Bokeh

Interactive charts — use `output_notebook()` for Bokeh in Colab

EDA

Inspect → Univariate → Bivariate → Correlation → Outliers

Data Cleaning

Handle nulls, duplicates, types, strings, outliers in a pipeline

Batch Processing

Process stored data in chunks — good for periodic reports

Stream Processing

Process data live using Python generators — low latency